Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Mission

Perform a comprehensive evaluation and evidence-led modernization of Stoquify's entire point-of-sale domain in:

`E:\ohada saas\Focused projects\stoquify`

Inspect the complete workflow, routes, server actions, services, schemas, hooks, UI components, integrations, permissions, operational procedures, and tests. Then implement the corrections and structural improvements needed to make POS demonstrably reliable, secure, understandable, efficient, and maintainable.

Treat this as an audit followed by implementation and verification. Do not stop at recommendations or visual cleanup.

Make daily work straightforward for cashiers, supervisors, store managers, inventory staff, finance and reconciliation teams, administrators, auditors, and support personnel. Map these personas to actual repository permissions; do not invent access rights.

Treat "enterprise grade" as measurable correctness, usability, resilience, and security requirements. Do not promise absolute security, zero defects, or unsupported compliance.

Review responsibilities

Activate the permanent reviewers for enterprise architecture; backend and integration engineering; database and migrations; application security, IAM and privacy; frontend and design systems; workflow UX, accessibility and localization; product and business processes; quality and release assurance; SRE and DevSecOps; and SaaS modularity, entitlement and product operations.

Also assess every domain reviewer: finance and internal controls; OHADA/SYSCOHADA and country-pack compliance; audit evidence and records governance; POS, inventory and offline consistency; purchasing and AP; HRIS and payroll; payments and settlement; accounting close and fiscal documents; analytics; AI/agent governance; APIs, webhooks and business events; and training, support and operational readiness.

Record a finding or a justified `not applicable` decision for every reviewer. Adjacent domains enter scope only through verified POS dependencies.

1. Establish current truth

Before editing:

1. Read applicable `AGENTS.md` instructions and relevant skills.
2. Record the dirty-worktree baseline and preserve existing changes, including untracked POS implementations.
3. Inventory all POS surfaces and their dependencies. Use `graphify-out/` to guide discovery, then verify relationships against current source.
4. Trace representative operations through:

   User action → component → hook → server action/API → domain service → database transaction → events/integrations → user-visible outcome.
5. Identify duplicated business rules, competing sources of truth, incomplete integrations, hidden dependencies, unsupported controls, and misleading readiness claims.
6. Capture reproducible findings before changing behavior. Distinguish implemented, partial, broken, unsupported, and intentionally deferred capabilities.

For each finding, record severity, affected role and journey, reproduction steps, source references, business impact, proposed correction, dependencies, and verification method.

2. Evaluate every business journey

For each journey, document its actor, prerequisites, authoritative state, inputs, approvals, side effects, success outcome, failure states, recovery path, and audit evidence.

Cover all implemented or exposed POS journeys:

- Terminal configuration, location selection, cashier access, register ownership, and shift opening.
- Catalog search, barcode scanning, item availability, pricing, tax, discounts, quantities, and cart updates.
- Walk-in and identified customers, customer-location compatibility, credit and on-account behavior where supported.
- Tender selection, exact cash, split payments, overpayment, change, rounding, and supported electronic payment evidence.
- Sale submission, duplicate clicks, interrupted requests, uncertain outcomes, replay protection, and recovery.
- Cart resume, parking, switching registers, reconnecting, changing cashiers, and returning in another tab or device.
- Receipt creation, printing and delivery where implemented, receipt access, revocation, and delivery failures.
- Returns, refunds, voids, overrides, and supervisor approvals where supported.
- Cash-drawer operations, shift reconciliation, shortages, overages, closing, and handover.

- Offline enrollment, local queues, replay, conflict review, revocation, and synchronization where implemented.
- Inventory movements, valuation, ledger posting, customer balances, payment reconciliation, reporting, and close controls.
- Operational support, exception investigation, configuration changes, and audit exports.

Do not expose a workflow as available merely because its enum, schema, button, or placeholder exists. Verify the complete execution path.

3. Correct architecture and business logic

Establish clear responsibilities:

- **Services** own business rules, authoritative calculations, state transitions, transactional work, and integration contracts.
- **Server actions and APIs** validate inputs, establish authenticated scope, enforce permissions and entitlements, call services, and return safe, consistent results.
- **Hooks** manage reads, mutations, cache scope, synchronization, cancellation, and reconciliation with authoritative results.
- **UI components** present state, collect input, explain decisions, and guide the next valid action.

Refactor oversized components or services only where evidence shows that decomposition improves correctness, testing, or maintainability. Preserve public contracts unless a coordinated change is necessary.

Specifically verify:

- Tenant-, location-, terminal-, cashier-, and shift-scoped reads and writes.
- Decimal monetary calculations, currency precision, tax rounding, and consistent totals.
- Atomic boundaries for sale, stock, payment, drawer, customer balance, and accounting effects.
- Concurrency protection against overselling, duplicate commits, conflicting cart edits, and shift-close races.
- Durable idempotency, request identity, payload validation, cart revisions, and replay semantics.
- Reliable event/outbox processing, provider retries, and post-commit failures.
- Cache invalidation and isolation across users, tenants, locations, terminals, and sessions.
- Preservation of drafts and entered values during failed mutations and refreshes.
- Database constraints, indexes, migration compatibility, and data provenance.

A receipt, notification, or cache failure after a successful commit must not misrepresent the sale as uncommitted or encourage duplicate submission.

4. Resolve checkout and recovery comprehensively

Use the recent checkout problems as regression scenarios, not as proof that the wider workflow is correct.

Define and document the actual lifecycle states, including:

- Ready, loading, saving, validating, and submitting.
- Empty cart or incomplete payment.
- Register owned elsewhere or ownership expired.

- Cart requiring resume or revalidation.
- Uncertain, pending, recorded, changed, or conclusively uncommitted attempts.
- Recoverable errors, denied operations, offline conflicts, and closed shifts.

For every state, specify what the user can do, why other actions are unavailable, and how the state changes.

Require that:

- A disabled Charge button explains the actual blocker and provides the appropriate nearby recovery action.
- "No result found" is never treated as proof that an attempt cannot still complete.
- A successful cart review replaces stale warning text and obsolete retry controls.
- Reviewing or checking an attempt never submits a sale.
- A retry or reviewed submission obeys the server's original-request and duplicate-prevention contracts.
- Ownership changes and reconnects cannot silently invalidate safety assumptions.
- Unknown attempts remain recoverable across reloads and supported device transitions.
- Recovery does not depend on clearing browser storage, deleting evidence, bypassing a lock, or manually forcing a successful state.
- The interface distinguishes "cart reviewed," "ready to submit," and "sale recorded."

Correct the underlying state and contract problems, rather than merely hiding warning banners.

5. Strengthen security and controls

Perform a concrete POS threat and abuse review.

Verify server-side authorization for every sensitive operation, including direct action/API calls that bypass the UI. Cover:

- Cross-tenant and cross-location access.
- Forged resource identifiers, prices, taxes, totals, tender evidence, and ownership claims.
- Unauthorized discounts, refunds, voids, drawer access, receipt access, and terminal changes.
- Stale sessions, revoked permissions, disabled modules, expired leases, replay, and concurrent requests.
- Appropriate fresh authentication, supervisor approval, and segregation of duties where supported or required.
- Safe errors, redacted logs, secret handling, receipt-token protection, and customer privacy.
- Tamper-resistant audit evidence and traceability to original business records.
- Provider authenticity and validated payment evidence; browser assertions must not establish that payment occurred.

Do not invent compliance badges or widen access to make a workflow pass. For statutory or country-specific changes, require current authoritative sources and qualified review.

6. Modernize the user experience

Deliver a coherent, professional POS experience aligned with Stoquify's design system.

Prioritize:

- Clear workflow order, readable hierarchy, consistent terminology, and a prominent next action.
- Fast keyboard and scanner use, effective touch controls, and predictable focus.
- Clear distinctions between pending, blocked, reviewed, completed, and failed operations.
- Useful loading, empty, denied, stale, offline, partial-success, and retry states.
- Preservation of user input, with explicit handling when resumed carts change prices, stock, customer context, or totals.
- Role-appropriate information and approval requests.
- Complete English and French localization, including validation, recovery guidance, accessible names, dates, and money.
- Responsive layouts, zoom behavior, contrast, focus visibility, screen-reader announcements, and non-color status cues.

Preserve the requested checkout layout: cart and payment content should expand naturally without a cramped nested vertical scrollbar. Use normal page scrolling when necessary and keep the path to Charge clear.

Avoid unsupported controls, decorative complexity, unexplained technical messages, and success claims that exceed server evidence.

Execution checklist

1. Establish the baseline and inventory.
2. Produce the workflow map, architecture map, threat model, and prioritized findings.
3. Define state contracts and correctness invariants before redesigning affected UI.
4. Fix critical authorization, integrity, transaction, concurrency, and recovery defects first.
5. Implement focused structural refactoring, then hook and UI improvements.
6. Verify each completed slice with meaningful positive, negative, concurrency, and recovery tests.
7. Validate integrated journeys in an isolated test environment.
8. Produce rollout, rollback, support, and training notes.
9. Report completed work, residual risks, and concrete blockers without overstating readiness.

Continue independently through authorized local work. Ask only for missing business decisions or consequential actions that cannot be resolved from repository evidence.

Evidence to inspect

Inspect current source and tests under:

- `actions/pos/`
- `services/pos/`
- `hooks/posHooks/`
- `components/pos/`
- `lib/pos/`
- `app/[locale]/(dashboard)/dashboard/pos/`
- `app/api/receipts/`
- Relevant terminal, drawer, sales, inventory, finance, accounting, security, entitlement, and offline boundaries.

- Relevant `prisma/`, `config/`, `scripts/`, localization, and browser-test files.
- Relevant `graphify-out/`, `what-next/`, and `innovation/` artifacts.

Include these recent reports:

- `what-next/pos-charge-button/2026-09-23/verification.md`
- `what-next/pos-checkout-layout/2026-09-23/verification.md`
- `what-next/pos-recovery-state/2026-09-23/verification.md`

Review the checkout lifecycle hook, checkout services, retained-request storage, cart revision logic, and commit-recovery tests explicitly.

Discover all actual routes and verify their English and French variants. Include `/dashboard/pos`, `/dashboard/settings/terminals`, `/dashboard/finance/cash-drawer`, and any additional verified POS surfaces.

Verification commands and evidence

Inspect scripts before running them, including their database and environment side effects.

Use appropriate focused checks:

- `npm run typecheck`
- `node node_modules/eslint/bin/eslint.js <changed-source-files>`
- `node node_modules/jest/bin/jest.js --runInBand --runTestsByPath <verified-test-files>`
- `npm run prisma:validate` when schema or database contracts are affected.
- Applicable POS, offline replay, inventory, payment, receipt, service-boundary, and workflow-assurance gates.
- `npm run build:app` for final build verification after focused checks.

Do not use `npm run build` indiscriminately: its current script includes migration deployment.

Run real PostgreSQL integration tests for transaction, locking, revision, and concurrency claims. Use a disposable, explicitly identified test database and safe provider simulations. Mock-only tests cannot establish financial transaction correctness.

Verify duplicate submission, delayed responses, response loss after commit, ownership contention, reconnects, changed carts, stale prices, insufficient stock, permission revocation, cross-tenant attempts, receipt failure, and shift-close races.

Browser-check all inventoried localized POS routes at representative desktop, tablet, and mobile sizes. Include keyboard, zoom, accessibility, long-cart, split-tender, and recovery states.

Save command, environment, revision, exit status, and results. Distinguish passed, failed, skipped, timed out, and blocked checks. Do not claim a successful build when packaging fails.

Expected artifacts

Create a dated run directory under:

`what-next/pos-enterprise-modernization/<run-date>/`

Include:

- `baseline.md`
- `workflow-and-architecture.md`
- `findings-register.md`
- `security-and-controls.md`
- `state-and-contracts.md`
- `implementation-summary.md`
- `verification-matrix.md`
- `rollout-and-support.md`
- `evidence/` containing relevant logs, test results, screenshots, and sanitized traces.

Each corrected finding must link to its implementation and verification evidence. Preserve unresolved findings with severity, impact, and the next required action.

Risk controls and non-goals

Preserve existing user changes and concurrent work. Do not reset the repository, overwrite unrelated modifications, remove tests to obtain a pass, or clean up unrelated lint warnings.

Do not perform live charges, refunds, voids, destructive database operations, broad migrations, production deployment, or security-access changes as incidental verification.

Do not rebuild unrelated modules, introduce speculative payment methods or AI features, replace frameworks without evidence, or perform repository-wide formatting.

Keep schema and data changes backward-compatible where possible. Provide migration and rollback evidence for necessary changes.

Success criteria

The work is complete when:

- Every inventoried POS journey has a verified implementation or an explicit, justified limitation.
- Critical and high-severity in-scope defects are corrected, or precisely identified as blocked with required action.
- Supported financial operations preserve authorization, precision, stock integrity, auditability, and duplicate protection under tested failure and concurrency conditions.
- Hooks and UI accurately represent authoritative states and provide clear recovery paths.
- Cashiers and other authorized roles can complete their tasks without unexplained dead ends.
- English and French experiences are coherent, responsive, and accessible within the verified scope.
- Required checks pass, and any failed or unavailable checks are clearly disclosed.
- The final report distinguishes implemented changes, verified outcomes, residual risks, and release limitations.

Conclude with a concise account of what changed, which user problems it resolves, how correctness was demonstrated, and what remains necessary before broader rollout.